DETECCIÓN Y RESOLUCIÓN DE CONFLICTOS DE CÓDIGO

Merge Conflicts

Documentación de los **diez ejercicios** de detección y resolución de conflictos de código.
Ejecución real, procedimiento aplicado y resultado verificado.

10/10	57	0	49
ejercicios	comprobaciones	fallos	comandos explicados

FASES DEL PROCESO

1. **Detectar** — estado y archivos afectados
2. **Entender** — qué quiso cada lado
3. **Resolver** — una decisión por bloque
4. **Verificar** — sin marcas y con pruebas
5. **Prevenir** — ramas cortas, integrar seguido

10 ejercicios paso a paso · Mapa mental en Canvas HTML5 · 51 capturas reales

01 Marco conceptual
02 Metodología
03 Procedimiento
04 Ejercicios 01–10
05 Anexo A · Comandos
06 Resultados y conclusiones

Un conflicto no es un error: es una decisión pendiente.

Ivan <75749748@iegb.edu.pe> · Octubre 2026 · equipo IVAN

github.com/75749748-blip/conflictos-git-ejercicios

1 · MARCO CONCEPTUAL

Conceptos básicos

Definiciones necesarias para interpretar el estado del repositorio y las marcas de conflicto.

01

El conflicto no es un fallo

Git detiene la operación para no perder trabajo. El repositorio permanece intacto y la decisión queda pendiente.

02

La decisión no es automática

Git muestra las dos versiones y unas marcas. La elección corresponde a quien mantiene el código.

03

Resolver equivale a decidir

Cabe conservar un lado, combinar los dos o eliminar el archivo. El resultado debe tener sentido para el proyecto.

DEFINICIONES BÁSICAS

Rama

Una línea de trabajo paralela. `main` suele ser la principal.

Merge

Une dos ramas en un commit nuevo que conserva las dos historias.

Marcador

`<<<<<<<`, `=======` y `>>>>>>>` separan las versiones en conflicto.

Índice

Zona donde se prepara lo que entrará en el próximo commit mediante `git add`.

Requisitos del entorno: Windows 11, Git para Windows 2.56.0 y una terminal Git Bash.

```
<<<<<<< HEAD
linea 2 (main)
=======
linea 2 (feature)
>>>>>>> feature
```

Anatomía de las marcas

- `HEAD` identifica la rama actual.
- `=======` separa las dos versiones.
- `>>>>>>> feature` nombra la rama que se integra.

Las marcas no se eliminan antes de leerlas: primero se decide y después se limpia el archivo.

CRITERIO GENERAL

Un cambio que no se comprende no debe resolverse: primero se consultan las dos versiones.

2 · METODOLOGÍA

Modelo de resolución en cinco fases

Las cuatro primeras fases componen el ciclo de resolución; la quinta agrupa las prácticas que evitan su repetición.



1 Detectar
Estado, archivos UU y marcas en el archivo.

2 Entender
Qué quería cada lado y por qué.

3 Resolver
Una decisión por cada bloque.

4 Verificar
Sin marcas, con pruebas y commit.

5 Prevenir
Ramas cortas, avisos e integración frecuente.

CRITERIO GENERAL

Un cambio que no se comprende no debe resolverse: primero se consultan las dos versiones.

Mapa elaborado con **Canvas HTML5** (`mapa-mental/mapa-mental.html`), disponible también como PNG de alta resolución.

3 · PROCEDIMIENTO

Secuencia de resolución

El orden es constante. Si el análisis no es concluyente, se regresa al primer paso antes de editar.

1

Detectar

Se consulta el estado y se localizan los archivos marcados.

`git status --short`

2

Entender

Se comparan las dos versiones antes de editar.

`git diff`

3

Resolver

Se elige o se combina; se eliminan todos los marcadores.

`editor + git add`

4

Verificar

Se confirma que no quedan marcas y que las pruebas pasan.

`git diff --check`

5 · Prevenir Ramas cortas, integraciones frecuentes y aviso cuando dos personas trabajan sobre el mismo archivo.

LOS 10 EJERCICIOS DOCUMENTADOS

#	Ejercicio	Operación	Resultado
01	**Conflicto simple en una línea**	merge	**5 PASS · 0 FAIL**
02	**Varios conflictos en un archivo**	merge	**6 PASS · 0 FAIL**
03	**Conflicto modificar / eliminar**	merge	**4 PASS · 0 FAIL**
04	**Conflicto añadir / añadir**	merge	**5 PASS · 0 FAIL**
05	**Conflicto con un archivo binario**	merge	**4 PASS · 0 FAIL**
06	**Conflicto durante un rebase**	rebase	**5 PASS · 0 FAIL**
07	**Conflicto en un cherry-pick**	cherry-pick	**5 PASS · 0 FAIL**
08	**Conflicto al recuperar un stash**	stash pop	**4 PASS · 0 FAIL**
09	**Inspección y herramienta de merge**	mergetool	**4 PASS · 0 FAIL**
10	**Estrategias automáticas**	estrategias	**15 PASS · 0 FAIL**

Alcance del documento. Cada ejercicio se documenta con su descripción, el procedimiento aplicado, la captura de consola y el resultado verificado. El anexo final reúne los comandos empleados, con su función y el motivo de su uso.

ESTADOS DE CONFLICTO EN GIT STATUS

UU

Los dos modificaron el archivo.

AA

Los dos añadieron el mismo archivo.

UD

Uno modificó y el otro eliminó.

DU

Uno eliminó y el otro modificó.

MERGE

Conflicto simple en una línea

DESCRIPCIÓN DEL CASO

Las dos ramas modificaron la misma línea de `app.txt`. Git no puede decidir cuál conservar y deja ambas versiones dentro del archivo, entre las marcas `<<<<<<<`, `=======` y `>>>>>>>`.

PROCEDIMIENTO APLICADO

1. El archivo se presenta con el estado **UU**: ambos lados lo modificaron.
   `git status --short`

2. Se conservan las dos líneas, **linea 2 (main)** y **linea 2 (feature)**, y se eliminan las marcas.
   `app.txt`

3. Se marca el archivo como resuelto y se confirma el merge.
   `git add + git commit`

5 PASS | **0** FAIL

Árbol de trabajo limpio. app.txt conserva los dos cambios y no quedan marcadores.

GITCAP01 - conflicto simple

```
================================================================
 Ejercicio : 01-conflicto-simple.sh
 Equipo    : IVAN   Usuario: Ivan
 Fecha     : 2026-10-28 22:57:13
 Git       : git version 2.56.0.windows.1
 Identidad : Ivan <75749748@iegb.edu.pe>
================================================================

workspace: /c/Users/gambo/Documents/Git-Conflictos/.work/ejercicio-01-linea

### PREPARACION ###

$ git init -b main
Initialized empty Git repository in C:/Users/gambo/Documents/Git-Conflictos/.work/ejercicio-01
linea/.git/

$ git config user.name Ivan

$ git config user.email 75749748@iegb.edu.pe

$ git config merge.conflictStyle zdiff3

$ printf 'linea 1\nlinea 2\nlinea 3\n' > app.txt

$ git add app.txt

$ git commit -m "base: crea app.txt"
[main (root-commit) 31f66b4] base: crea app.txt
 1 file changed, 3 insertions(+)
 create mode 100644 app.txt

$ git switch -c feature
Switched to a new branch 'feature'

$ printf 'linea 1\nlinea 2 (feature)\nlinea 3\n' > app.txt
```

Captura real 1 de 4 · consola del equipo

5 PASS · 0 FAIL

MERGE

Varios conflictos en un archivo

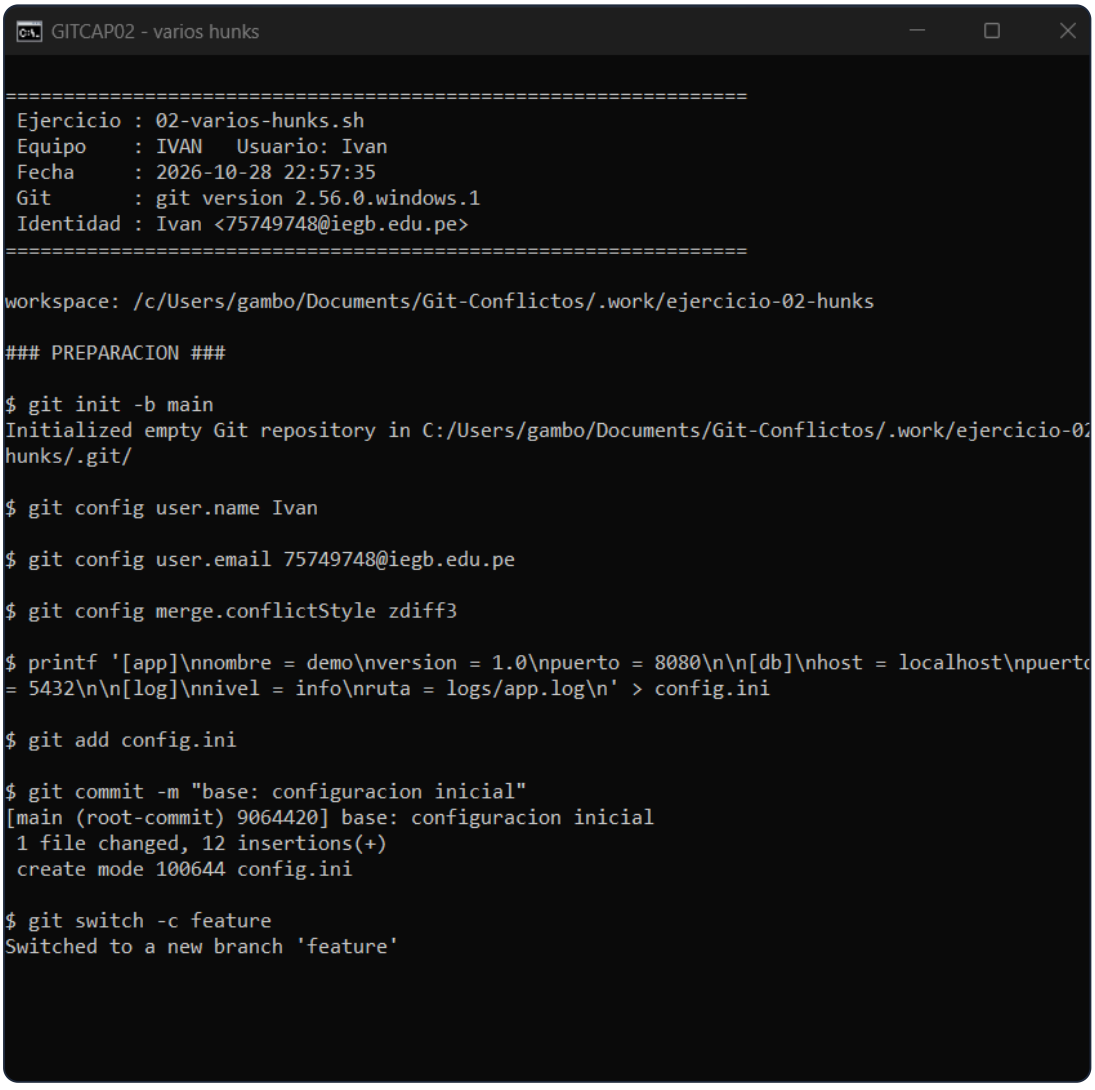
DESCRIPCIÓN DEL CASO

El mismo `config.ini` presenta dos zonas en conflicto: el nombre de la aplicación y el nivel de log. Cada zona admite una decisión independiente; no conviene aplicar una sola respuesta a todo el archivo.

PROCEDIMIENTO APLICADO

1. Se localizan los dos bloques de conflicto y se identifica el cambio de cada lado.
   `git diff`
2. El nombre se unifica como **demo (main+feature)** y el nivel de log se fija en **debug**.
   `config.ini`
3. Se eliminan las marcas y se confirma el merge.
   `git add + git commit`

6 PASS | **0** FAIL — Árbol de trabajo limpio. Las dos zonas quedaron resueltas con decisiones independientes.



Captura real 1 de 5 · consola del equipo — 6 PASS · 0 FAIL

MERGE

Conflicto modificar / eliminar

DESCRIPCIÓN DEL CASO

Una rama editó `notas.txt` y la otra lo eliminó. El caso equivale a decidir si el archivo debe seguir existiendo. Es el estado **UD** y la decisión es de producto, no de sintaxis.

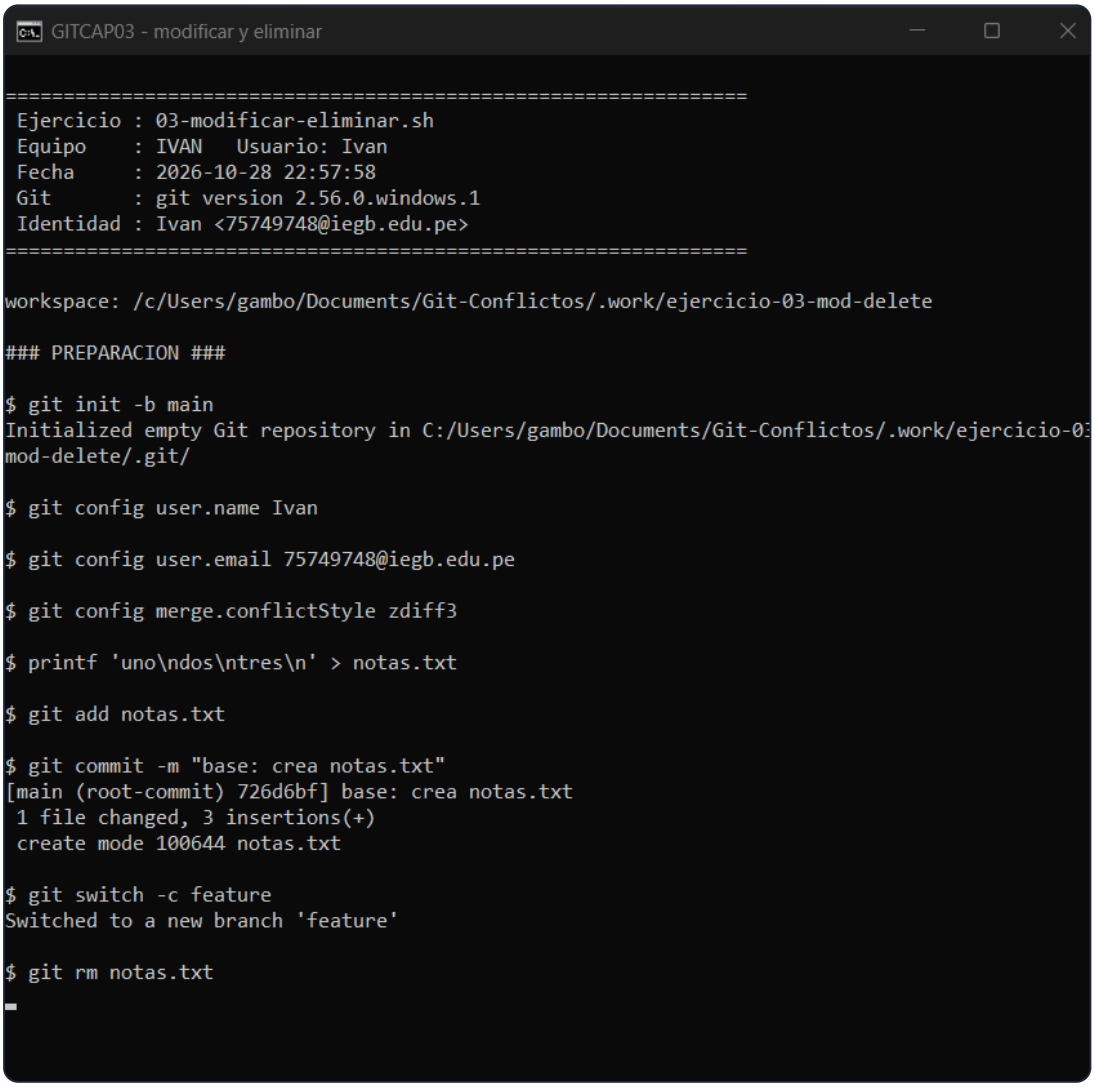
PROCEDIMIENTO APLICADO

1. Se comprueba el estado y se lee el contenido que una de las ramas quiso conservar.
   `git status + cat`

2. La decisión se registra con **add**, si se conserva el archivo, o con **rm**, si se acepta la eliminación.
   `git add · git rm`

3. Se confirma el merge con la decisión adoptada.
   `git commit`

4 PASS | **0** FAIL — Árbol de trabajo limpio. Se documentó la solución de eliminar notas.txt.



Captura real 1 de 5 · consola del equipo — 4 PASS · 0 FAIL

MERGE

Conflicto añadir / añadir

DESCRIPCIÓN DEL CASO

Dos ramas crearon el mismo archivo nuevo, `CHANGELOG.md`, con contenidos distintos. Es el estado **AA**: no existe una versión base del archivo que sirva de referencia.

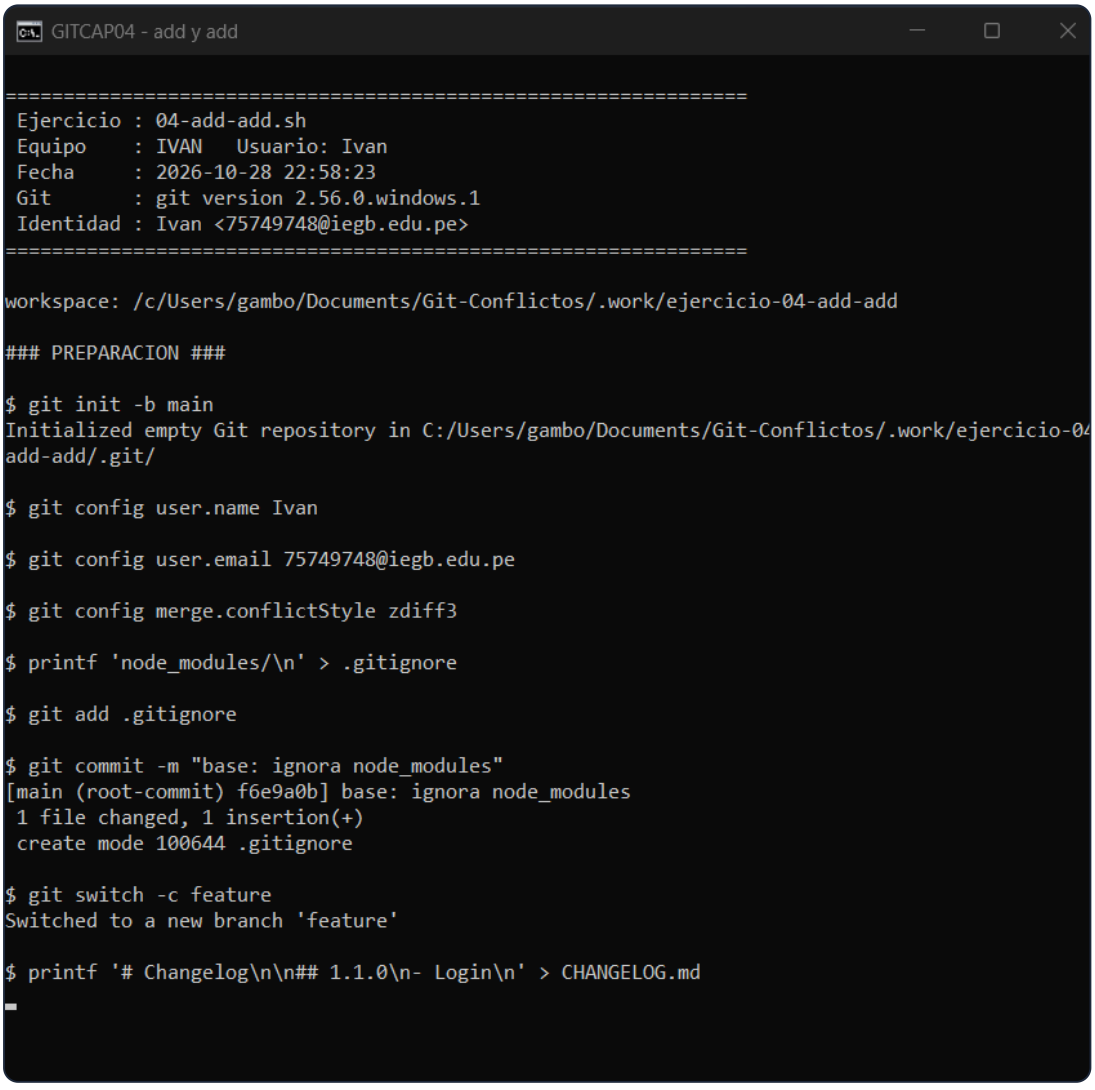
PROCEDIMIENTO APLICADO

1. Git concatena los dos documentos. Ambos se leen antes de editar.
   `cat CHANGELOG.md`

2. Las entradas **1.1.0** y **1.0.1** se ordenan en un único changelog.
   `CHANGELOG.md`

3. Se guarda el archivo unificado y se confirma.
   `git add + git commit`

5 PASS | **0** FAIL — Árbol de trabajo limpio. Un solo CHANGELOG con las dos entradas.



Captura real 1 de 4 · consola del equipo — 5 PASS · 0 FAIL

MERGE

Conflicto con un archivo binario

DESCRIPCIÓN DEL CASO

`logo.bin` es un archivo binario. Git no puede comparar su contenido ni mostrar marcas: solo puede conservar una versión completa. Para mantener las dos, una de ellas tendría que renombrarse antes de integrar.

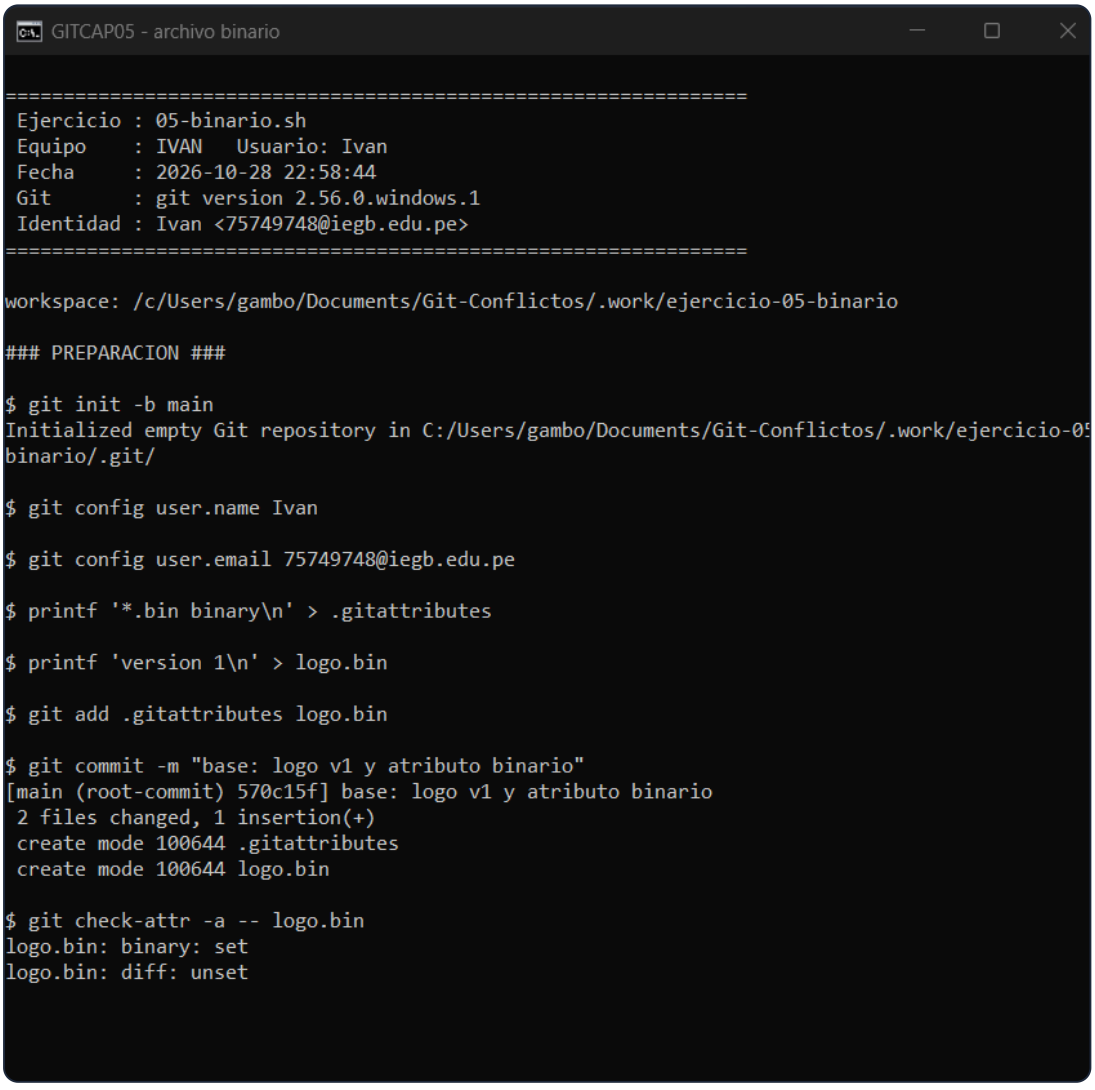
PROCEDIMIENTO APLICADO

1. Git muestra el aviso **Cannot merge binary files**; no hay texto que editar.
   `git diff`

2. Se selecciona una versión completa del archivo (en este caso, la de **feature**).
   `git checkout --theirs`

3. Se marca la versión elegida y se confirma el merge.
   `git add + git commit`

4 PASS | **0** FAIL — Árbol de trabajo limpio. logo.bin conserva la versión de feature.



Captura real 1 de 4 · consola del equipo — 4 PASS · 0 FAIL

REBASE

Conflicto durante un rebase

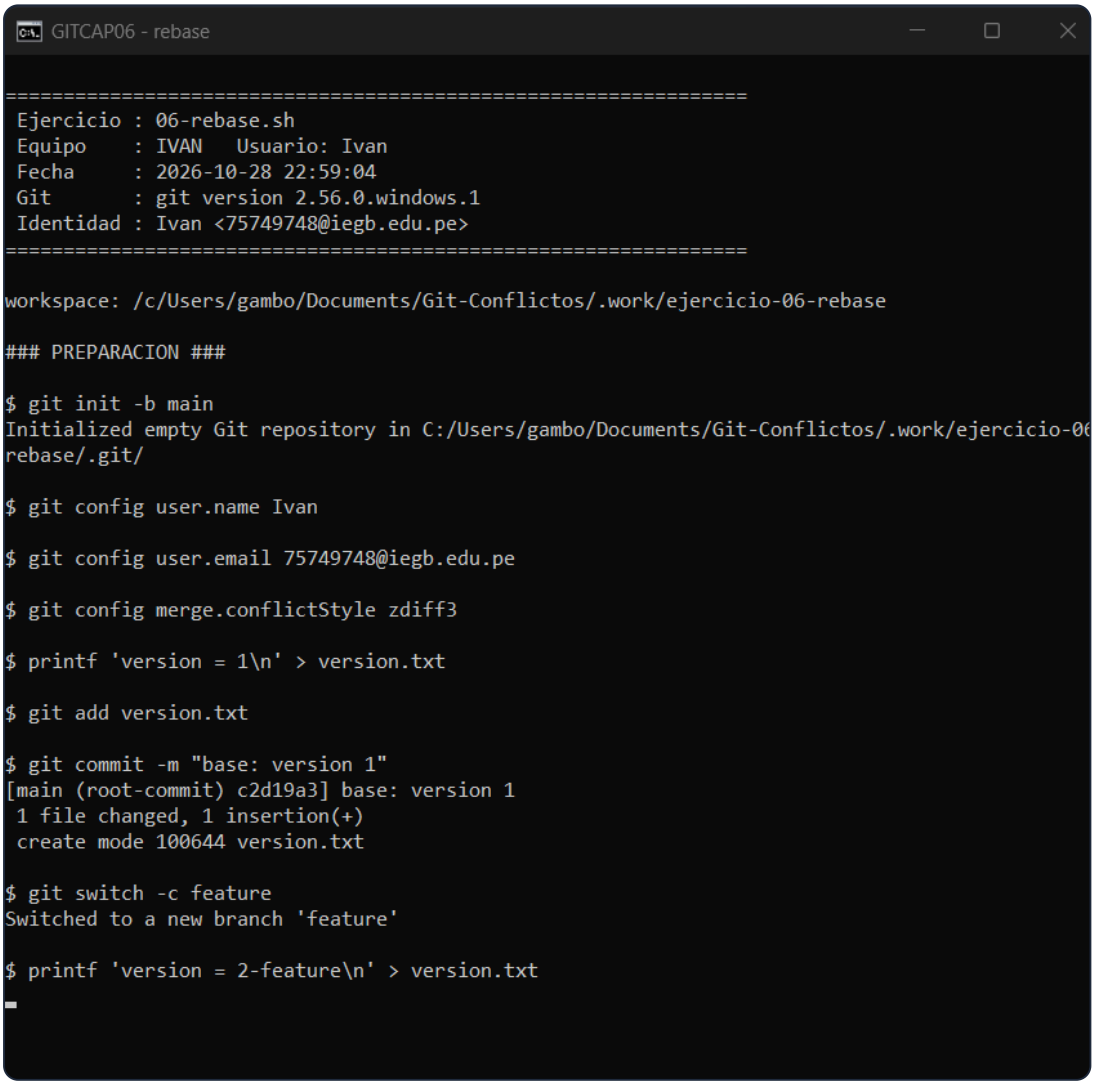
DESCRIPCIÓN DEL CASO

En un rebase, Git reaplica los commits sobre otra base. Si el parche no encaja, **ours** es la rama base y **theirs** es el commit propio: al revés que en un merge.

PROCEDIMIENTO APLICADO

1. Se identifica el archivo que detuvo el rebase.
   `git status`
2. Se comparan la base (main) y el commit (feature) y se resuelve el archivo.
   `git show :2: :3:`
3. Se marca la resolución y se reanuda el rebase.
   `git add + git rebase --continue`

5 PASS | **0** FAIL — Historia lineal, sin commit de merge, con los cambios conservados.



Captura real 1 de 4 · consola del equipo — 5 PASS · 0 FAIL

CHERRY-PICK

Conflicto en un cherry-pick

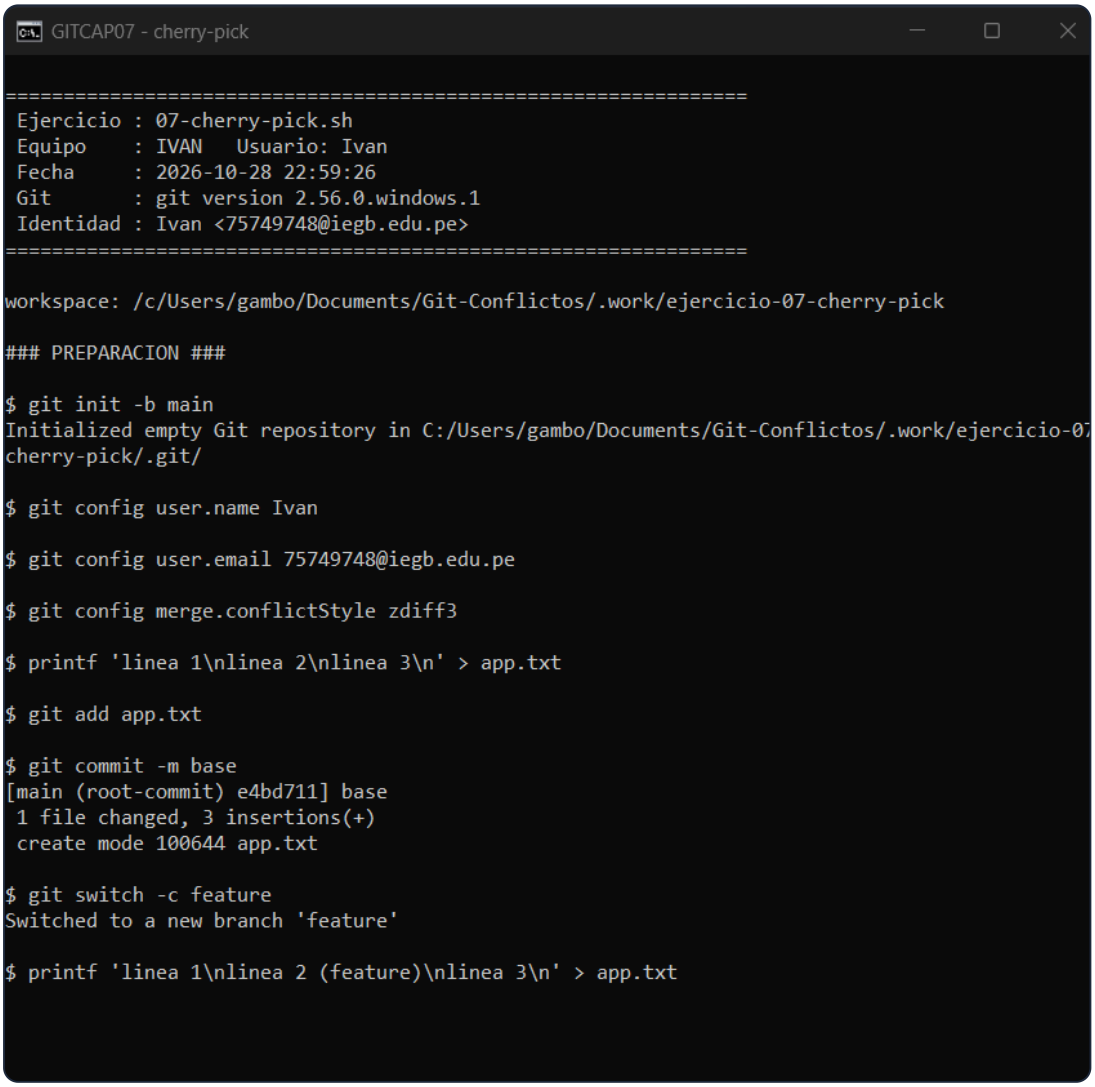
DESCRIPCIÓN DEL CASO

Cherry-pick copia un commit concreto de `main` sobre `feature`. El parche ya no encaja en la nueva base y Git se detiene para que se ajuste.

PROCEDIMIENTO APLICADO

1. Se identifica el commit de main que se va a aplicar.
   `git log --oneline`
2. El commit se aplica sobre feature y se registra el conflicto.
   `git cherry-pick`
3. Se resuelve app.txt, se marca y se reanuda el cherry-pick.
   `git add + --continue`

5 PASS | **0** FAIL — Árbol de trabajo limpio. El cambio se aplicó una sola vez, sin duplicarlo.



Captura real 1 de 5 · consola del equipo

5 PASS · 0 FAIL

STASH POP

Conflicto al recuperar un stash

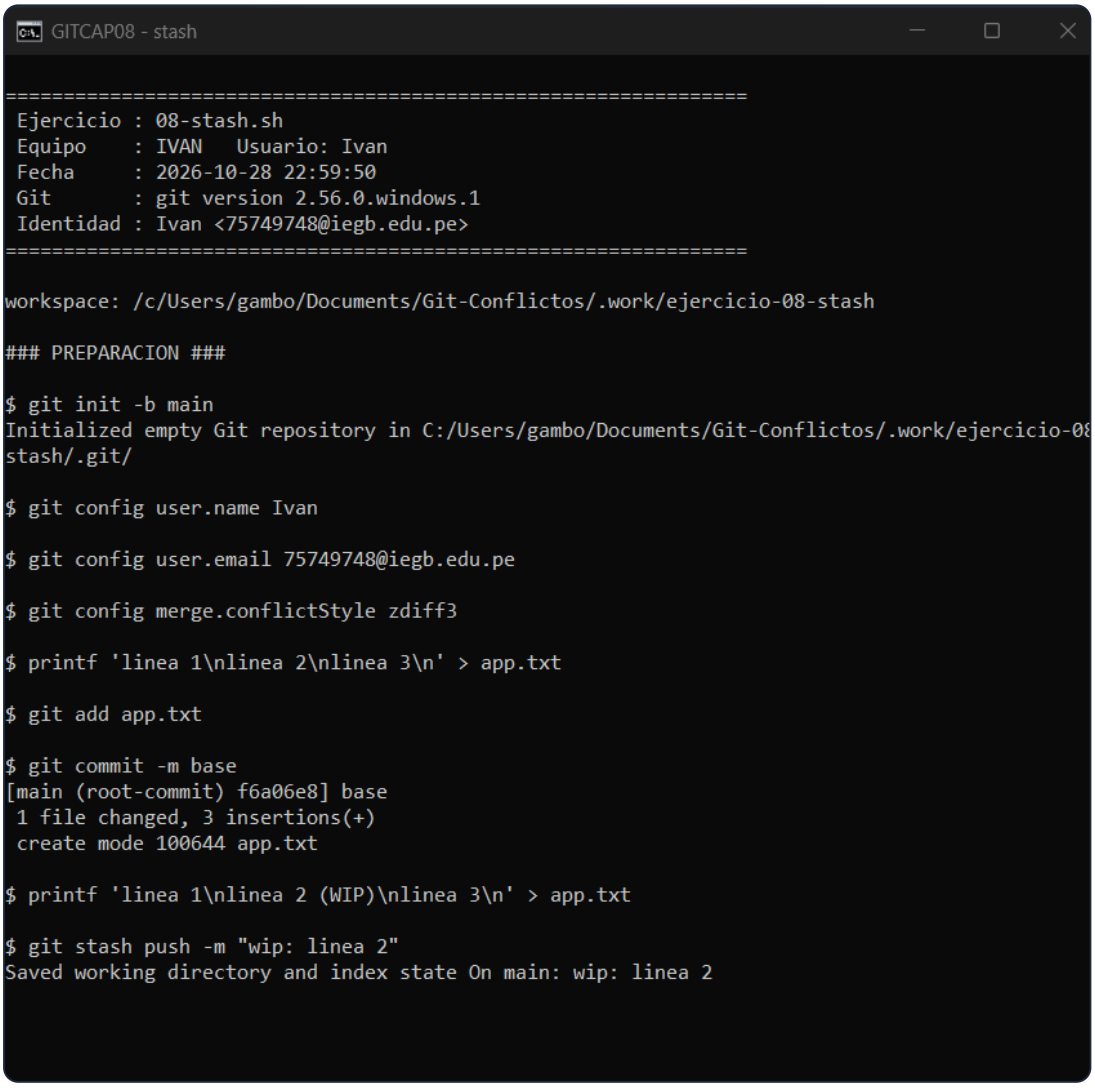
DESCRIPCIÓN DEL CASO

El trabajo a medias se guardó con `stash`, la rama principal avanzó y, al recuperarlo, entra en conflicto con el commit nuevo. Las marcas muestran **Updated upstream** y **Stashed changes**.

PROCEDIMIENTO APLICADO

1. Se recupera el trabajo guardado y se registra el conflicto.
   `git stash pop`
2. El WIP se combina con el cambio de main y se eliminan las marcas.
   `app.txt`
3. Se marca la resolución, se descarta el stash ya aplicado y se confirma.
   `git add + stash drop`

4 PASS | **0** FAIL — Árbol de trabajo limpio. El WIP se recuperó y la lista de stash quedó vacía.



Captura real 1 de 4 · consola del equipo — 4 PASS · 0 FAIL

MERGETOOL

Inspección y herramienta de merge

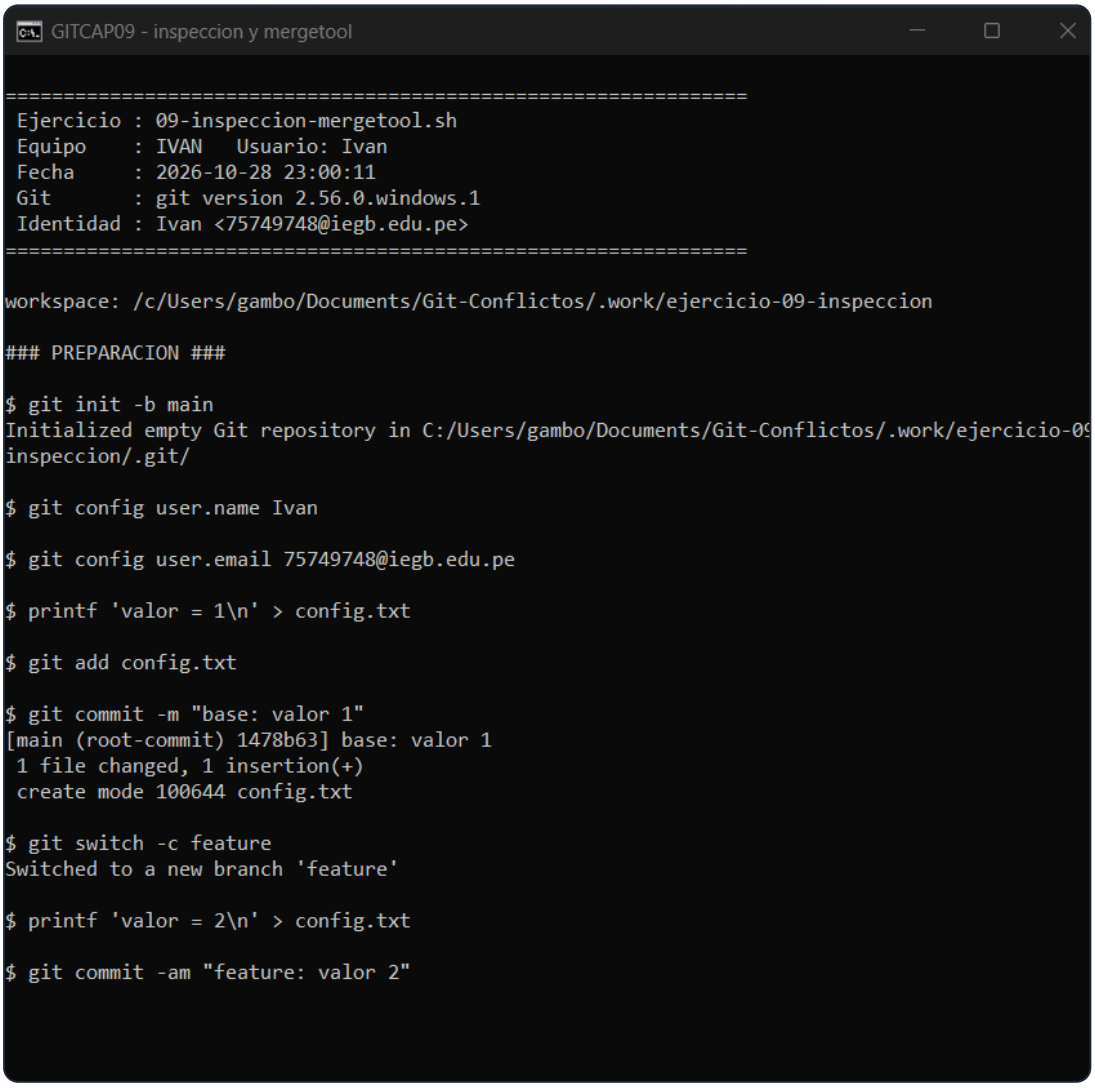
DESCRIPCIÓN DEL CASO

Antes de editar conviene reconstruir las tres versiones: la base (`:1:`), la versión local (`:2:`) y la remota (`:3:`). Con esa información, la herramienta de merge permite elegir el resultado sin adivinar.

PROCEDIMIENTO APLICADO

1. Se leen la base, la versión local y la versión remota.
   `git show :1: :2: :3:`
2. La herramienta de merge construye el resultado final de config.txt.
   `git mergetool`
3. Se guarda, se marca y se confirma el merge.
   `git add + git commit`

4 PASS | **0** FAIL — Árbol de trabajo limpio. config.txt se resolvió con información completa.



Captura real 1 de 6 · consola del equipo — 4 PASS · 0 FAIL

ESTRATEGIAS

Estrategias automáticas

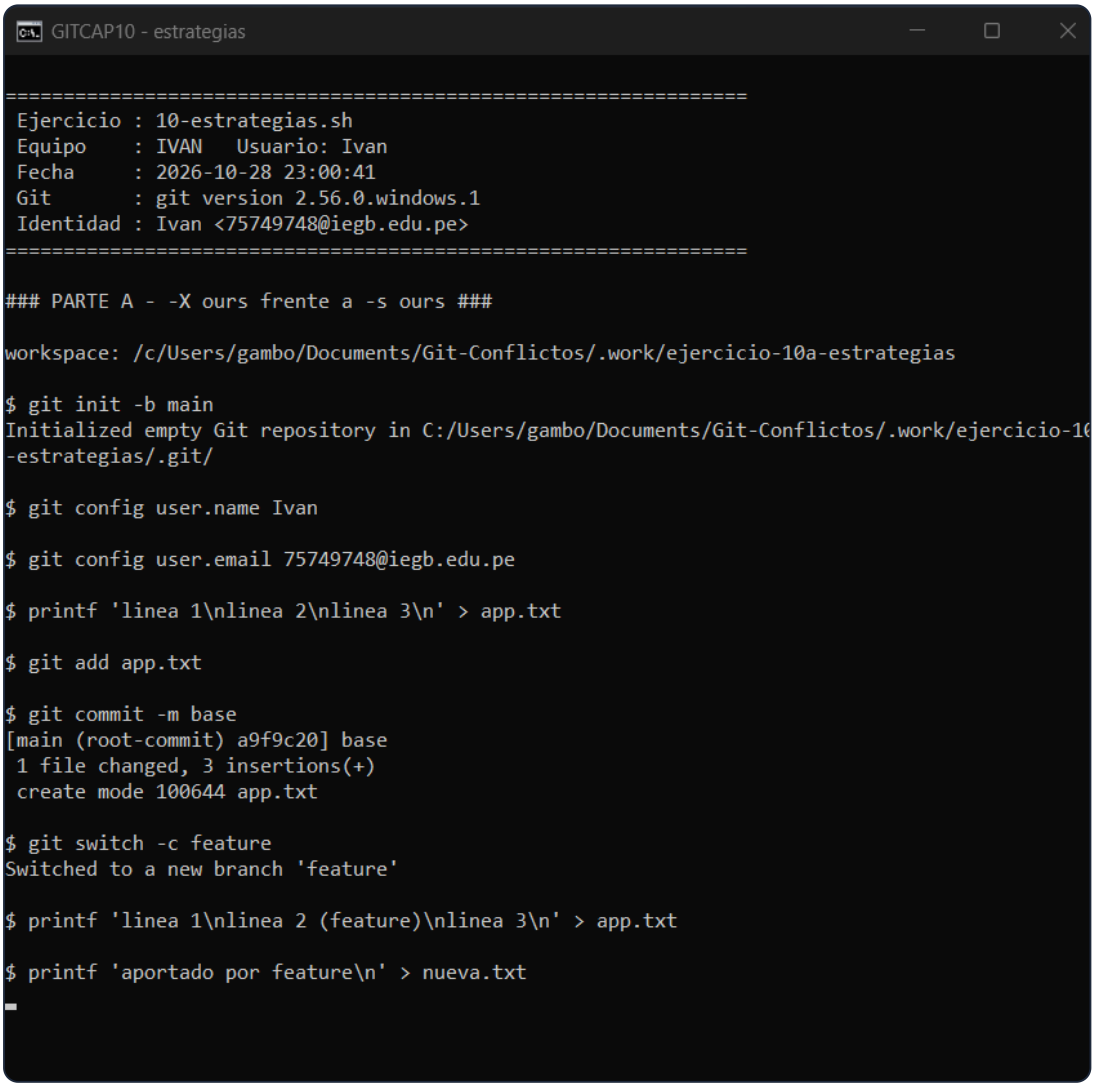
DESCRIPCIÓN DEL CASO

Git ofrece atajos para casos previsibles: `-X ours`, `-X theirs`, un driver `merge=ours` y la estrategia `union`. Reducen trabajo, pero no sustituyen la revisión del resultado.

PROCEDIMIENTO APLICADO

1. Ambas estrategias se prueban en ramas desechables y se comprueba qué se conserva.
   `-X ours · -X theirs`
2. Se configura un driver para archivos de lock que no deben cambiar.
   `merge=ours`
3. Se conservan las dos líneas cuando ambas aportan información y se valida el resultado.
   `union`

15 PASS | **0** FAIL — Árbol de trabajo limpio. Cada estrategia produjo el efecto esperado.



Captura real 1 de 10 · consola del equipo — 15 PASS · 0 FAIL

49 COMANDOS

Comandos empleados

49 comandos. Se indica la función de cada uno y el motivo de su empleo en los ejercicios. Todos aparecen en las capturas de las páginas anteriores.

COMANDO	FUNCIÓN Y MOTIVO DE USO
CREAR EL REPOSITORIO Y FIRMAR LOS COMMITS	
`git init -b main`	**Función.** Crea un repositorio nuevo en la carpeta actual y llama main a la rama principal. **Motivo de uso.** Cada ejercicio empieza de cero en su propia carpeta, así el conflicto se reproduce igual siempre.
`git config user.name "Ivan"`	**Función.** Guarda el nombre que Git escribirá como autor de cada commit. **Motivo de uso.** Los commits de las capturas corresponden al autor del documento y no a un usuario de ejemplo.
`git config user.email "75749748@iegb.edu.pe"`	**Función.** Guarda el correo que Git asocia a cada commit. **Motivo de uso.** Es el correo de la cuenta de GitHub: así el commit y la cuenta quedan vinculados.
CREAR RAMAS Y GUARDAR CAMBIOS	
`git switch -c feature`	**Función.** Crea la rama feature y se cambia a ella en un solo paso. **Motivo de uso.** Así el cambio que provoca el conflicto se hace en una rama aparte y main queda intacta.
`git switch main`	**Función.** Vuelve a la rama main, con los archivos tal como estaban allí. **Motivo de uso.** Sirve para hacer el cambio opuesto en main y que las dos ramas choquen al unirlas.
`git add app.txt`	**Función.** Marca el archivo para que entre en el próximo commit. **Motivo de uso.** Git solo guarda lo que se marca: primero se elige, después se confirma.
`git commit -m "base: crea app.txt"`	**Función.** Guarda una foto de los cambios marcados, con un mensaje que la describe. **Motivo de uso.** Cada paso del ejercicio queda en la historia con un mensaje que dice qué hizo.
`git commit -am "feature: cambia la linea 2"`	**Función.** Hace las dos cosas a la vez: marca los archivos ya conocidos y los confirma. **Motivo de uso.** Es un atajo para no escribir add y commit por separado cuando el archivo ya existía.
VER QUÉ ESTÁ PASANDO	
`git status`	**Función.** Muestra la rama actual, qué archivos cambiaron y si hay un merge a medias. **Motivo de uso.** Describe el estado exacto en el que quedó el repositorio.
`git status --short`	**Función.** La misma información condensada en una línea por archivo, con dos letras de estado. **Motivo de uso.** Permite registrar el estado completo del conflicto en una sola captura.
`git diff`	**Función.** Muestra línea por línea qué cambió respecto a la última versión guardada. **Motivo de uso.** Permite ver las dos versiones enfrentadas antes de decidir cuál conservar.
`git diff --check`	**Función.** Busca marcas de conflicto olvidadas y espacios raros al final de línea. **Motivo de uso.** Es el control final: si queda un <<<<<<< sin borrar, este comando lo delata.
`git log --oneline --graph --all --decorate`	**Función.** Dibuja la historia de todas las ramas en una línea por commit. **Motivo de uso.** Sirve para comprobar que el merge o el rebase dejaron la historia como se esperaba.
`git ls-files -u`	**Función.** Lista solo los archivos que están en conflicto, con sus tres versiones internas. **Motivo de uso.** Es la forma rápida de saber cuántos archivos siguen sin resolver.
ENTENDER EL CONFLICTO POR DENTRO	
`git show :1:app.txt`	**Función.** Muestra la versión base: el archivo tal como era antes de que las ramas se separaran. **Motivo de uso.** Es el punto de partida común; sin él no se entiende qué cambió cada lado.
`git show :2:app.txt`	**Función.** Muestra la versión local, la de la rama en la que se está trabajando. **Motivo de uso.** En un merge es main; en un rebase es la rama base. Conviene saber cuál es cuál.

49 COMANDOS

Comandos empleados

COMANDO	FUNCIÓN Y MOTIVO DE USO
`git show :3:app.txt`	**Función.** Muestra la versión del otro lado, la que llega con el merge. **Motivo de uso.** Comparar :2: con :3: es la manera más limpia de decidir qué conservar.
`git show HEAD:app.txt`	**Función.** Muestra cómo quedó el archivo en el último commit de la rama actual. **Motivo de uso.** Confirma que la resolución se guardó de verdad, no solo en el editor.
`git diff --base -- config.txt`	**Función.** Compara el archivo actual contra la versión base. **Motivo de uso.** Señala qué aportó cada lado respecto al punto común, sin el ruido de las marcas.
`git log --merge --oneline -- notas.txt`	**Función.** Lista solo los commits de las dos ramas que tocaron ese archivo. **Motivo de uso.** Dice quién cambió qué y en qué orden, útil cuando el conflicto no es obvio.
RESOLVER Y CERRAR	
`git add app.txt`	**Función.** Marca el archivo como resuelto una vez editado. **Motivo de uso.** Mientras no se marque, Git sigue considerando que el conflicto está abierto.
`git rm notas.txt`	**Función.** Borra el archivo del disco y avisa a Git de que la decisión es eliminarlo. **Motivo de uso.** Es la respuesta correcta cuando una rama editó el archivo y la otra lo eliminó.
`git checkout --theirs -- logo.bin`	**Función.** Trae el archivo completo tal como está en la otra rama, sin mezclar. **Motivo de uso.** Un binario no se puede mezclar por líneas: hay que elegir una versión entera.
`cat app.txt`	**Función.** Imprime el contenido del archivo en la consola. **Motivo de uso.** En las capturas muestra el contenido del archivo después de resolver.
TERMINAR UNA OPERACIÓN QUE QUEDÓ A MEDIAS	
`git rebase --continue`	**Función.** Retoma el rebase desde donde se detuvo, ya con el conflicto resuelto. **Motivo de uso.** Un rebase no termina solo: hay que decirle que siga tras cada choque.
`git cherry-pick --continue`	**Función.** Retoma la copia del commit que quedó a medias. **Motivo de uso.** Cherry-pick también se pausa: el commit no se aplica hasta que se resuelve.
`env GIT_EDITOR=true git rebase --continue`	**Función.** Continúa sin abrir el editor de texto para aceptar el mensaje por defecto. **Motivo de uso.** Permite que el ejercicio avance solo, sin que nadie tenga que pulsar teclas.
`git reset --hard HEAD~1`	**Función.** Deshace el último commit y descarta los cambios del árbol de trabajo. **Motivo de uso.** Se usa en ramas desechables para probar una estrategia y volver atrás sin ensuciar.
LAS ÓRDENES QUE PROVOCAN EL CONFLICTO	
`git merge feature`	**Función.** Trae la rama feature a la rama actual y crea un commit que une las dos historias. **Motivo de uso.** Es la operación base del taller: junta dos líneas de trabajo sin perder ninguna.
`git rebase main`	**Función.** Reaplica los commits propios encima de main, uno por uno, como si se hubieran escrito después. **Motivo de uso.** Deja una historia lineal, pero cambia los hashes y puede repetir el mismo conflicto varias veces.
`git cherry-pick fbd46ff`	**Función.** Copia un commit concreto de otra rama y lo aplica aquí. **Motivo de uso.** Sirve para trasladar un solo arreglo sin arrastrar toda la rama.
`git stash push -m "wip: linea 2"`	**Función.** Guarda los cambios sin confirmar en una entrada aparte y deja el árbol limpio. **Motivo de uso.** Permite cambiar de rama con trabajo a medias sin perderlo.
`git stash pop`	**Función.** Recupera el último trabajo guardado y lo vuelve a aplicar sobre el árbol. **Motivo de uso.** Al recuperarlo, si main cambió mientras tanto, aparece el conflicto.

49 COMANDOS

Comandos empleados

COMANDO	FUNCIÓN Y MOTIVO DE USO
`git stash list`	**Función.** Muestra las entradas guardadas, con su número y su mensaje. **Motivo de uso.** Confirma que el stash se creó y, al final, que el cajón quedó vacío.
`git stash drop`	**Función.** Elimina la entrada guardada que ya no es necesaria. **Motivo de uso.** Se usa cuando el cambio ya se aplicó y no se quiere conservar la copia.
ATAJOS Y AUTOMATIZACIÓN	
`git merge --no-edit -X ours feature`	**Función.** Hace el merge y, en cada choque, conserva la línea local sin preguntar. **Motivo de uso.** Demuestra que el atajo funciona, pero también que puede perder cambios del otro lado.
`git merge --no-edit -s ours feature`	**Función.** Une la historia pero ignora por completo el contenido de la otra rama. **Motivo de uso.** Se compara con -X ours: aquí no se mezcla nada, el resultado es idéntico al de main.
`git config merge.ours.driver true`	**Función.** Define un driver que, ante un conflicto, deja siempre la versión local. **Motivo de uso.** Se aplica a archivos que no deben cambiar nunca, como los de configuración propia.
`git check-attr -a -- logo.bin`	**Función.** Muestra qué atributos de Git afectan a ese archivo. **Motivo de uso.** Comprueba que el archivo está declarado como binario antes de resolverlo.
`git config rerere.enabled true`	**Función.** Activa el «recuerdo» de resoluciones: Git apunta cómo resolviste un choque. **Motivo de uso.** Si el mismo conflicto reaparece, lo resuelve solo con la decisión anterior.
`git config rerere.autoUpdate true`	**Función.** Hace que rerere deje el archivo ya resuelto y marcado. **Motivo de uso.** Ahorra repetir a mano la misma resolución en cada intento del rebase.
`git rerere status`	**Función.** Lista los archivos cuyo conflicto rerere está siguiendo. **Motivo de uso.** Muestra que la función está activa y qué está memorizando.
`git config merge.tool resolver`	**Función.** Le dice a Git qué programa abrir cuando pides ayuda para resolver. **Motivo de uso.** Permite usar una herramienta propia en lugar de editar el archivo a mano.
`git mergetool`	**Función.** Inicia la herramienta de merge configurada y guía por cada archivo en conflicto. **Motivo de uso.** Es la alternativa visual a editar las marcas dentro del archivo.
`git config mergetool.prompt false`	**Función.** Evita que Git pregunte si quieres abrir la herramienta en cada archivo. **Motivo de uso.** Sin esto, la ejecución se quedaría esperando una tecla y no podría capturarse.
COMANDOS DE APOYO DEL TALLER	
`printf 'linea 1\nlinea 2\n' > app.txt`	**Función.** Escribe ese texto en el archivo app.txt, reemplazando lo que hubiera. **Motivo de uso.** Crea el contenido exacto que después provoca el conflicto, sin depender de un editor.
`ls nueva.txt`	**Función.** Comprueba si el archivo existe en la carpeta. **Motivo de uso.** Verifica si el merge trajo o no un archivo que solo existía en la otra rama.
`rm -f resolver.sh config.txt.orig`	**Función.** Borra archivos temporales sin pedir confirmación. **Motivo de uso.** Limpia la herramienta de prueba para que el ejercicio pueda repetirse igual.
`git diff HEAD~1 HEAD --stat`	**Función.** Resume qué archivos cambió el último commit, sin mostrar el contenido. **Motivo de uso.** Confirma de un vistazo que el merge tocó lo que debía tocar.

5 · RESULTADOS

Resultados de la ejecución

Cada cifra procede de la última línea del registro de ejecución: RESULTADO: N PASS, 0 FAIL.

10/10	57	0	51
ejercicios completados	comprobaciones PASS	fallos	capturas reales

#	Ejercicio	PASS	FAIL	Estado
01	Conflicto simple en una línea	5	0	✓ Verificado
02	Varios conflictos en un archivo	6	0	✓ Verificado
03	Conflicto modificar / eliminar	4	0	✓ Verificado
04	Conflicto añadir / añadir	5	0	✓ Verificado
05	Conflicto con un archivo binario	4	0	✓ Verificado
06	Conflicto durante un rebase	5	0	✓ Verificado
07	Conflicto en un cherry-pick	5	0	✓ Verificado
08	Conflicto al recuperar un stash	4	0	✓ Verificado
09	Inspección y herramienta de merge	4	0	✓ Verificado
10	Estrategias automáticas	15	0	✓ Verificado

Criterio de verificación. Cada script genera el conflicto, aplica la resolución y ejecuta comprobaciones automáticas. En los diez casos el árbol de trabajo quedó limpio y sin marcadores.

OPERACIONES CUBIERTAS

merge · rebase · cherry-pick · stash pop · binarios · mergetool · estrategias

OBTENCIÓN DE LA EVIDENCIA

1. **Generación del conflicto**
   Un script prepara un repositorio temporal y provoca el conflicto.
2. **Resolución y validación**
   Se aplica la solución y se ejecutan comprobaciones PASS/FAIL.
3. **Registro de la evidencia**
   Las capturas de consola y el registro completo se incorporan al repositorio.

6 · CONCLUSIONES

Procedimiento de contingencia

Actuación prevista cuando la resolución no es evidente.

Cuando la resolución no es clara

1. No se elimina el archivo ni la carpeta `.git`.
2. Se ejecuta `git status --short` para identificar los archivos en conflicto.
3. Se ejecuta `git diff` y se leen ambas versiones antes de editar.
4. Si el análisis no concluye, se aborta la operación: `git merge --abort`, `git rebase --abort` o `git cherry-pick --abort`.
5. Abortar es una salida válida: la operación puede repetirse desde el inicio.

Errores que deben evitarse

- ✗ Confirmar un archivo que todavía contiene `<<<<<<<`.
- ✗ Aplicar `-X ours` o `-X theirs` sin revisar los cambios.
- ✗ Eliminar `.git` para resolver un conflicto.
- ✗ Resolver sin analizar, solo para que el comando finalice.

CUESTIÓN CENTRAL

¿Qué pretendía cada uno de los dos lados?

UBICACIÓN DE LA EVIDENCIA

REPOSITORIO PÚBLICO

github.com/75749748-blip/conflictos-git-ejercicios

`ejercicios/` scripts y logs · `build/real-captures/` capturas de consola · `mapa-mental/` canvas HTML5 y PNG · `documentacion/` este documento

LISTA DE COMPROBACIÓN PREVIA AL COMMIT

- ✓ Sin marcas pendientes `<<<<<<<`
- ✓ `git diff --check` sin errores
- ✓ Pruebas del proyecto superadas
- ✓ Cambio de cada lado comprendido

Un conflicto bien resuelto deja el historial más claro, no más sucio.

10/10 ejercicios · 57 PASS · 0 FAIL